

Lecture 2

*Lecturer: Nikhil Karamchandani**Scribe(s): Vedika Rathi, Shubhhi Singh*

1 Applications of Error Correcting Codes

Way back in the 1950s, when coding theory was in its initial stages, it was believed that it had utility in only communication systems (for which it was originally intended). But now, coding theory finds its applications in various diverse fields. Let's look at a few examples:

1.1 Coding for Storage

Considering the huge data produces nowadays, storing it at a single location is not a 'smart' decision. Moreover, if due to some failure, we lose the data which was stored at only one location, it would be impossible to retrieve it back. This calls for the need for using distributed storage systems, introducing redundancy and in turn having more than one copy of the data.

How is introducing redundancy relevant to error detection/correction?

Let's look at **Simple Parity Check Code** ($k = 2, n = 3$):

There are only 4 codeword messages : (00, 01, 10, 11), which would need just 2 bits to uniquely identify them, but we transmit one extra parity bit (which is the $\oplus$ of all the bits). Now, if due to channel error, one bit gets flipped, and we transmit 00 but 01 is received. Since we knew the parity of the transmitted message, we would know that there is some error. Thus, adding one extra bit of redundancy provides us with error detection up to 1 bit.

message	code
00	000
01	001
10	101
11	110

Table 1: Simple Parity Check Code

1.2 Network Coding

Here, we take an example of a butterfly network. Each edge is only able to carry one alphabet in one iteration. So, when the middle link receives both 'a' and 'b' from S1 & S2, it has two choices. Method 1 - it can send the alphabet 'a' in the first iteration, forward it to R1, and send 'b' in the next iteration. Now, if we make use of coding, we can send $a \oplus b$

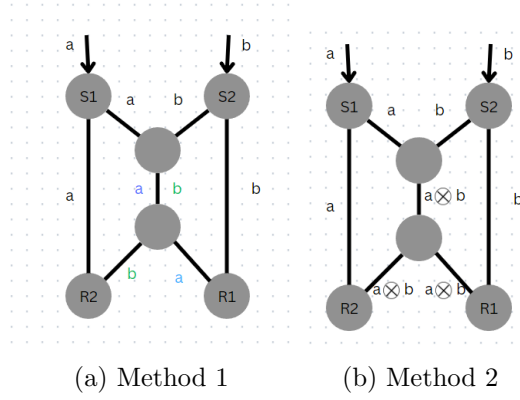


Figure 1: A simple butterfly network

over the middle link, and since both receivers are also directly connected to the source, they will be able to decode the actual alphabet.

Thus, we can see that coding improves **throughput** of systems.

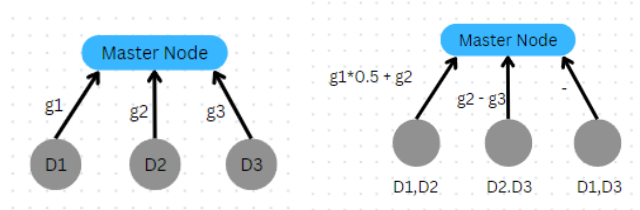
1.3 Coded Computation

Information is generally stored on distributed servers, and when it is required, several servers communicate with a master node and send the required information to it. It is then the job of the master node to interpret the complete information from the parts that it receives from the various servers. All servers do not have the same performance, and some of them might fail and/or lag behind (called stragglers). So, one must introduce redundancies in each server, so that if maximum n servers fail, the information is not lost.

Let's take a 'Gradient Coding' example. The master node has to minimize a loss function $L(x, y, \beta)$ over a set D .

$$D = D_1 \cup D_2 \cup D_3$$

Gradient $\sum \nabla L(x_i, y_i, \beta)$ over each subset are g_1, g_2, g_3 respectively. So, the master node needs the information $g_1 + g_2 + g_3$ from the servers.



(a) No redundancies (b) A combination of subsets

Figure 2: A Gradient Coding Example

If the servers have no overlap, as in (a) if even 1 fails, information is lost. Take (b) though,

even if server 3 fails, a combination of the information from servers 1 and 2, such as $[2 * (\text{Server 1 info}) + \text{Server 2 info}]$ would give $g_1 + g_2 + g_3$.

Thus, we see that coding can provide **robustness** in systems.

2 Block Code

Let us take a finite alphabet set - F . An (n, M) **block code** is a subset $C \subseteq F^n$ such that $|C| = M$. Here, n is the **code length**, C is the **code**, and elements of C are called **codewords**. k is defined as $\log_{|F|} M$ and **rate** $R = \frac{k}{n}$.

We can consider each codeword as a vector in the space F^n . The error correction and detection depend on the closeness of codewords/vectors. The distance would tell us how much error it could tolerate without getting confused. Now, to define “closeness”, we need a notion of distance between the codewords.

2.1 Hamming Distance

Let $x, y \in F^n$, $d_H(x, y)$ defined as the **Hamming Distance** is the number of positions where x and y differ.

Being a ‘measure of distance’, Hamming distance satisfies certain conditions:

- $d(x, y) \geq 0$ (It is non-negative)
- $d(x, y) = d(y, x)$ (It is symmetric)
- $d(x, y) + d(y, z) \geq d(x, z)$ (It satisfies Triangle Inequality)

2.2 Minimum Distance of Code C

The minimum distance of code C is interpreted as the minimum distance amongst all possible pairs of codewords in C . It is defined as $\min_{x, y \in C} d(x, y) = d(C)$

As we will see, the minimum distance of code plays a key role in determining the error correction/detection capabilities, and therefore the notation used to represent a code is (n, M, d) or (n, k, d) . Some examples are:

- $(3, 2, 3)$ repetition code over $F = \{0, 1\}$
2 messages, each of length 3 : $\{000, 111\}$
 $d = 3$ (The codewords differ at 3 positions)
- $(3, 4, 2)$ simple parity check code over $F = \{0, 1\}$
4 messages, each of length 3 : $\{000, 010, 101, 110\}$
 $d = 2$ (Any 2 codewords differ at 2 positions)

2.3 Decoder

Let C be an (n, M, d) code over an alphabet F and let S be a channel defined by the triplet (F, Φ, Prob) . A decoder for the code C with respect to the channel S is a function $D : \Phi^n \rightarrow C$.

3 Introduction to Error Correction

Claim: An (n, M, d) code can correct all error patterns which have at most $\lfloor \frac{d-1}{2} \rfloor$ errors.

Decoding Function:

$$D(y) = \arg \min_c d(c, y)$$

i.e. it will point us to the codeword closest to y .

Proof: Assume to the contrary.

$$t_x(\text{transmitted codeword}) = c$$

$$r_x(\text{received codeword}) = y \text{ s.t. } d(c, y) \leq \lfloor \frac{d-1}{2} \rfloor$$

yet, $\exists c' \neq c$ for which $d(y, c') \leq d(y, c)$.

Using the triangle inequality,

$$d(c, c') \leq d(c, y) + d(y, c') \leq \lfloor \frac{d-1}{2} \rfloor \cdot 2 \leq d-1$$

This is a contradiction since d is the minimum distance between any two codewords. Hence, proved. Intuitively, we can consider a space in which all codewords are separated by a

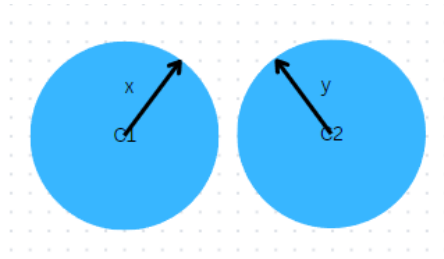


Figure 3: $x, y \leq \lfloor \frac{d-1}{2} \rfloor$

minimum distance d . If there are balls of radius $\leq \lfloor \frac{d-1}{2} \rfloor$ around each codeword, then they do not intersect each other. As long as y lies in only one of the balls, error correction would not be a problem.

4 Introduction to Error Detection

Claim: An (n, M, d) code can detect all error patterns which have at most $d-1$ errors.

Decoding Function:

$$D(y) = \begin{cases} c & \text{if } y = c \\ \text{error} & \text{o/w} \end{cases}$$

i.e. if y does not exactly belong to a codeword, we call it an error.

By logical understanding, unless the codeword transmitted (say, c) gets converted to another codeword received (say, c'), we would be able to detect errors.

The distance between two codewords is at least d . So up to $d-1$ errors, we shall be able

to detect an error.

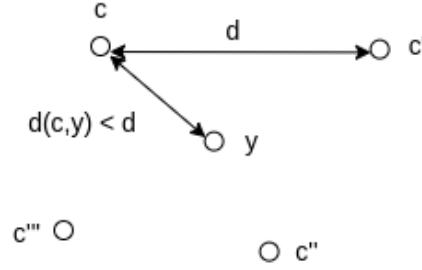


Figure 4: $d(c,y) \leq d-1$

5 Error Correction + Detection

Claim: $\exists t, l$ s.t. $2t + l \leq d - 1$

Suppose there is a (n, M, d) code. If the number of errors is $\leq t$, all error patterns will be corrected. Otherwise, if the number of errors $\leq t + l$, any error pattern will be detected.

Decoding Function:

$$D(y) = \begin{cases} c & \text{if } c \in C, d(y, c) \leq t \\ \text{error} & \text{o/w} \end{cases}$$

Proof:

For the first case,

$$t \leq \lfloor \frac{d-1}{2} \rfloor$$

$\implies$ error correction would always happen.

For the 2nd case, assume that,

$$t_x = c$$

$$r_x = y \text{ s.t. } d(y, c) \leq t + l$$

The error would not be detected if y lies within distance t of some $c' \neq c$ i.e $d(y, c') \leq t$. But from the triangle inequality,

$$d(c, c') \leq 2t + l \leq d - 1$$

which is a contradiction. Hence, proved.